

Accelerating Exact Unknot Recognition

Incremental preprocessing, modular obstructions,
and sparse cobordism elimination

Technical report prepared for Vladimir Reshetnikov

18 September 2026

Abstract

We modify the pure-Python recognizer in the supplied `Knots.zip/fast/` rather than substitute an unrelated package. The resulting implementation is complete when its optional resource limits are disabled, retains the scanning Khovanov backend over $\mathbb{F}_2$, and has no third-party runtime dependencies. The changes comprise incremental Reidemeister reductions and scan ordering, a linear-time descending-diagram test, two sound modular rejection tests, cached cobordism topology, characteristic-two algebraic shortcuts, and sparsity-aware Gaussian cancellation. The supplied 36-crossing benchmark now has reduced Khovanov rank 2949 computed in 2.23 seconds (median of three cold-cache runs); the unchanged baseline reaches our 30-second limit on the same host. This is a raw homology computation, not merely a prefilter verdict. All 37 tests pass, including independent cube calculations and differential checks. Some tiny inputs become slower, and the full benchmark tables include them. We prove asymptotic improvements for several subroutines, but do *not* claim a quasi-polynomial worst-case bound for the complete recognizer. We explain precisely why the hierarchy materials do not transfer such a bound to this implementation.

1 Deliverable and scope

The input is a validated, one-component planar knot diagram with n crossings. The package accepts the original planar-diagram (PD), closed-braid, and grid JSON formats. The result is `UNKNOT`, `KNOTTED`, or `UNKNOWN`. The last result means that a requested resource ceiling was reached, not that an invariant was trivial. With no time or Khovanov-object ceiling, the pipeline is a decision procedure. Internal Jones-filter ceilings merely abandon that filter and continue to the complete backend.

The implementation is in `fastunknot/`; the unmodified supplied Python package is preserved in `baseline/fastunknot/`. The files `diagram.py` and `alexander.py` are unchanged. The new `filters.py` contains the modular algorithms. The main changes to `scan.py`, `simplify.py`, and `recognize.py` are described below. Tests, fixed benchmark inputs, an isolated-process comparison driver, raw results, and the sources for this report are included. The existing MIT No Attribution license is retained. The archive manifest records file hashes; provenance metadata identifies the supplied archive and the unchanged sources.

This is not an implementation of Lackenby’s hierarchy algorithm. Its strongest achieved result is a substantial practical improvement to the supplied exact algorithm, together with genuine better asymptotic bounds for preprocessing and order construction. The worst-case envelope for the complete implementation remains $2^{\mathcal{O}(n)}$. The rank routine computes total rank and cube-degree ranks, not the full quantum-graded Poincaré polynomial or integral torsion.

2 What the hierarchy materials establish

The supplied talk’s main theorem slide, physical page 14, explicitly announces an $n^{\mathcal{O}(\log n)}$ unknot algorithm and labels the result “L 2021” [1]. The supplied July 2026 paper presents a hierarchy-based algorithm for incompressibility and obtains unknot recognition as a special case [2]. Its introduction describes further speed-up as future work. Section 9 deliberately gives only a crude iteration estimate; Section 10 refines the construction using interior parallelity bundles and controls hierarchy length in terms of the initial complexity. These are important distinctions: an announcement, a terminating algorithm, and a particular implementation with a proved running-time bound are different objects.

In the notation of Section 9, one relevant bound is

$$\text{number of iterations} \leq L(g+1)^L, \quad (1)$$

where L bounds hierarchy length and g bounds a surface complexity. A linear bound on L in an initial input-complexity parameter is not, by itself, a logarithmic bound. Nor is an iteration count a bit-complexity bound until representation sizes and the operations inside an iteration are controlled.

A quantitative implementation target. For the *crude bound* (1), the hypothetical conditions $L \leq a \log n$, $g \leq n^b$, and n^c work per iteration would imply

$$n^c L(g+1)^L \leq n^c a \log n (n^b + 1)^{a \log n} = \exp(\mathcal{O}((\log n)^2)).$$

This is a sufficient set of conditions, not a claim that these bounds have been established for the supplied hierarchy procedure, or that every route to quasi-polynomial time must use them. A different sufficient contract would be a recurrence

$$T(N) \leq N^a T(\rho N) + N^b, \quad 0 < \rho < 1,$$

with polynomial-size encodings and polynomially bounded branching. Its $\mathcal{O}(\log N)$ levels give the same quasi-polynomial envelope.

Turning either contract into code would require more than a normal-surface search called behind an opaque interface. A useful implementation boundary would consist of the following concrete data and operations: an incidence encoding of handle structures and boundary patterns; binary multiplicities for families of parallel normal pieces; splitting and regluing operations on these compressed families; recognition and removal of parallelity bundles; and hierarchical multi-surfaces with explicitly tracked complexity. Each operation needs an invariant explaining what manifold and boundary pattern its output represents. The balancing or Cheeger-region step must construct an admissible surface and prove the required decrease, rather than merely return a graph separator. Finally, the number of operations and the bit length of every multiplicity need bounds in the original n .

None of those geometric operations is supplied by the Khovanov engine. A small scan boundary, a small cache, or a successful numerical experiment cannot fill that gap. This work therefore improves the available engine instead of labeling an incomplete hierarchy interface a quasi-polynomial implementation.

3 Pipeline and its decision invariant

The optimized pipeline is

$$\begin{aligned} & \text{R1/R2 simplification} \longrightarrow \text{descending test} \\ & \longrightarrow \text{modular determinant} \longrightarrow \text{bounded modular bracket} \\ & \longrightarrow \text{exact Alexander polynomial} \longrightarrow \text{scanning Khovanov rank over } \mathbb{F}_2. \end{aligned}$$

The first two stages may establish triviality. The next three may establish nontriviality. Only the last stage is a complete invariant test for this purpose. A modular residue that matches the unknot is never interpreted as an unknot verdict. In particular, the implementation does not assume that the Jones polynomial detects the unknot.

Theorem 3.1 (Decision rule). *Assume the input satisfies the package’s knot-diagram validation conditions and the implemented cobordism operations represent the stated Khovanov chain operations. Every completed pipeline verdict is sound. In the absence of user-imposed time and object limits, the pipeline terminates with a verdict.*

Proof. Crossing-decreasing Reidemeister moves preserve the represented knot, and a descending diagram represents the unknot. Sections 5 and 6 prove the modular implications. The inherited exact Alexander stage rejects only a normalized polynomial different from 1.

For the final stage, Shumakovitch’s splitting over $\mathbb{F}_2$ implies

$$\dim_{\mathbb{F}_2} \text{Kh}(K; \mathbb{F}_2) = 2 \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2)$$

[5, Corollary 3.2.C]. Universal coefficients imply that the rational reduced rank is at most the reduced rank over $\mathbb{F}_2$; the former is nonzero. Kronheimer–Mrowka’s theorem identifies reduced rational rank one with the unknot [4]. Thus reduced $\mathbb{F}_2$ rank one implies triviality, while the unknot itself has reduced $\mathbb{F}_2$ rank one.

The finite cube of resolutions and exact finite-dimensional algebra give a terminating fallback. The bounded Jones stage cannot prevent reaching it. Every earlier successful stage makes a sound decision, so the composition has the same property. $\square$

This theorem states the mathematical contract, not a machine-checked proof of every Python line. The implementation is validated by the independent and differential tests in Section 10. The JSON evidence contains the input digest, simplification trace, residues or polynomial, and, when applicable, scan order and rank data. A homology result is a reproducible computation, not a short independently verified certificate.

4 Incremental preprocessing

4.1 A mutable dart involution for R1 and R2

Give each crossing four permanent dart identifiers $4c, \dots, 4c + 3$. The involution α pairs darts along edges, while $\sigma(4c + i) = 4c + (i + 1 \bmod 4)$ rotates counterclockwise around a crossing. Faces are orbits of $F = \sigma\alpha$. Therefore a removable monogon is recognized by $F(d) = d$. A candidate bigon satisfies $F^2(d) = d$ with its two darts at distinct crossings; the additional parity test checks that one strand is over at both crossings. Odd ports are over-passes in the inherited PD convention.

The old implementation searches all faces and rebuilds the complete diagram after each move. The new one stores legal local candidates in a heap. When one or two crossings are removed, their

opposite ports are joined through the deleted region. At most eight deleted darts need inspection; only a constant number of external involution entries change. Every newly created monogon or bigon must meet that changed neighborhood. Rechecking all four darts of every affected external crossing therefore finds every newly legal move. Stale heap entries are discarded on extraction.

Heap keys use move kind followed by permanent crossing identifiers, retaining the old R1-before-R2 and lexicographic policy. Permanent identifiers retain relative order after deletion. A Fenwick tree converts them to indices in the *current* diagram for the original replayable `Move` format. This avoids rebuilding or renumbering the surviving diagram after every move. A final reconstruction calls the original validator once.

Proposition 4.1. *The incremental simplifier produces the same move sequence as the baseline policy, up to harmless edge-label renaming, and uses $\mathcal{O}(n \log n)$ word operations and $\mathcal{O}(n)$ space.*

Proof. The initial candidates are exhaustive. Locality of the face successor map establishes the candidate invariant after every deletion. Permanent ordering agrees with current ordering, and the Fenwick prefix sum gives its exact current rank; hence each chosen valid move is the baseline’s first legal move. There are at most n deletions, each creates a bounded number of heap entries, and both heap and prefix-sum work cost $\mathcal{O}(\log n)$ per operation. The initial and final passes are at most $\mathcal{O}(n \log n)$. $\square$

Here “word operations” treats normalized labels and indices of $\mathcal{O}(\log n)$ bits as machine words. The statement does not assert constant cost for arbitrary-precision integers of unbounded length. The old repeated full-diagram work has a quadratic lower bound on the family of successive curls used in the preprocessing benchmark.

4.2 Descending diagrams by interval coverage

Fix an oriented traversal with $2n$ visits. For a crossing let o be its over-visit position and u its under-visit position. The cyclic starting positions for which this crossing is encountered underneath first form exactly the interval $(o, u]$. Add all such intervals to a circular difference array. A zero in the prefix sum is a start for which every crossing is first encountered over. Repeat for the reverse orientation.

Proposition 4.2. *A descending basepoint, when one exists in either traversal direction, is found in $\mathcal{O}(n)$ time and space.*

Proof. A start is valid exactly when it belongs to none of the forbidden intervals. Each interval is represented by at most two ordinary intervals, hence a constant number of difference-array updates. Two prefix scans and two traversals suffice. The returned dart identifies the actual basepoint; the algorithm is not merely a boolean relaxation. $\square$

5 A modular determinant before polynomial arithmetic

The inherited Alexander implementation computes an exact polynomial using fraction-free elimination. That is useful but unnecessary for many negative instances. At $t = -1$, a Wirtinger Fox row has the form

$$2x_{\text{over}} - x_{\text{under},1} - x_{\text{under},2} = 0.$$

Over-passing edge segments are joined by disjoint-set union to construct the Wirtinger arcs. Remove one row and one column of the resulting matrix, and compute its determinant in $\mathbb{F}_p$, with the fixed

prime $p = 1\,000\,000\,007$. Sparse dictionaries represent rows; among candidate pivots in a column the algorithm picks a row with the fewest nonzero entries. Row swaps and pivot products are retained, so the computation is a determinant, not just a rank test.

Proposition 5.1. *If the computed residue is neither 1 nor -1 modulo p , the input knot is nontrivial.*

Proof. The Fox minor evaluates to $\Delta_K(-1)$ up to sign; multiplication of the Alexander polynomial by $\pm t^k$ also gives only a sign at $t = -1$. For the unknot the normalized Alexander polynomial is 1, so every allowed signed value is ± 1 . Reduction modulo p preserves equality. A different residue therefore contradicts triviality. $\square$

This filter is deterministic and one-sided. A nontrivial determinant can be congruent to ± 1 ; that is an inconclusive result, not an error. The worst-case cost is $\mathcal{O}(n^3)$ field operations and $\mathcal{O}(n^2)$ stored field elements. The prime is fixed, so coefficient swell is eliminated at this stage. The exact polynomial remains available for cases that pass both modular filters.

6 A bounded, independent Jones-bracket engine

6.1 Normalization and soundness

For ports (a, b, c, d) in the inherited counterclockwise PD convention, use smoothings $0 = ((a, b), (c, d))$ and $1 = ((a, d), (b, c))$ with coefficients A and A^{-1} . Set $\delta = -A^2 - A^{-2}$. It is convenient to count *every* closed circle, including the first, giving the all-loops bracket

$$B_D(A) = \sum_{s \in \{0,1\}^n} A^{n-2|s|} \delta^{c(s)}, \quad (2)$$

where $c(s)$ is the number of circles in state s and an empty PD denotes one crossingless circle. This differs by a factor δ from the common normalization assigning bracket 1 to a single circle.

The writhe $w(D)$ is computed from the incoming ports of *both* strands. If $u \in \{0, 2\}$ and $o \in \{1, 3\}$ are the incoming under- and over-ports, the sign used here is $+1$ when $o - u \equiv 3 \pmod{4}$ and -1 otherwise. The inherited `Diagram.signs()` is appropriate to its local Fox-row convention; it is not a safe substitute for this writhe computation after arbitrary half-turns of PD rows. That function is left unchanged to avoid altering the Alexander backend.

A positive curl contributes $-A^3$: indeed, $A\delta^2 + A^{-1}\delta = -A^3\delta$. The other curl contributes $-A^{-3}$. Thus the relevant unknot identity is

$$B_D(A) = \delta(-A^3)^{w(D)} \quad \text{when } D \text{ represents the unknot.} \quad (3)$$

This is the bracket normalization underlying the Jones invariant [6]. To see directly why no sign is being guessed, in the local Temperley–Lieb algebra write $R = A1 + A^{-1}e$ and $R^{-1} = A^{-1}1 + Ae$. Then $RR^{-1} = 1 + (A^2 + A^{-2} + \delta)e = 1$, proving the R2 identity. The relations $e_i^2 = \delta e_i$ and $e_i e_{i+1} e_i = e_i$ give the R3 identity; the two curl calculations give R1 after writhe normalization.

Evaluate (2) at $A = 2$ in $\mathbb{F}_p$. A discrepancy from (3) proves knottedness. The code never divides by δ ; A and $-A^3$ are nonzero, so negative exponents are well-defined. The same routine supports any integer $A \not\equiv 0 \pmod{p}$ for testing. There is no probability of a false positive from modular arithmetic. A collision can only cause the filter to pass the knot on to later stages.

6.2 Frontier dynamic programming

At each crossing, the frontier consists of edges with exactly one processed endpoint. A smoothing state of the processed part induces a perfect matching of these boundary labels; every closed component is already accounted for by a factor δ . The table maps each canonical matching to the sum of its coefficients in $\mathbb{F}_p$.

For each table entry and each smoothing, an independent disjoint-set computation joins the old matching arcs to the two new smoothing arcs. A connected component with no remaining boundary endpoints contributes a factor δ ; a component with two endpoints contributes a new matching pair. Components with any other number of endpoints signal an invariant violation. Equal output pairings are merged by field addition and zero coefficients are removed. Unlike the Khovanov routine, this implementation uses no dotted-cobordism composition, no delooping matrices, and no homology cancellation. Its independent structure is useful for testing.

Proposition 6.1. *After every scan prefix, each table coefficient is exactly the sum of partial bracket-state weights inducing that boundary matching. Therefore the final empty-matching coefficient is (2) modulo p , independently of the scan order.*

Proof. Initially the empty matching has coefficient one. Attaching a crossing partitions full extensions uniquely by their previous state and one of the two local smoothings. The union-find computation detects precisely the circles closed by that attachment and the surviving pairs. Multiplying by the local smoothing coefficient and δ for each such circle gives the correct extended weight. Summing identical matchings is valid because all future attachments depend only on those matchings. Induction completes the proof; the crossingless input is handled separately as one circle. $\square$

If the maximum frontier size is w , the number of possible pairings is at most $(w - 1)!!$, with $(-1)!! = 1$. Thus a conservative uncapped bound is

$$\mathcal{O}(n \text{ poly}(w) (w - 1)!!) = n 2^{\mathcal{O}(w \log w)}$$

field operations, plus scan-order construction. An arbitrary processed region need not be a disk with all endpoints on one cyclic boundary, so we do not silently replace this bound by a Catalan number. The default limits are 50 000 live states and 1 000 000 transitions. Reaching either is an instruction to skip this filter, not to decide the knot.

The Conway and Kinoshita–Terasaka examples both have Alexander polynomial one in the supplied data and are accepted by the first modular filter. The new bracket test rejects both without computing their reduced Khovanov rank 33. Their raw residues and scan statistics are included in the benchmark output. This is a genuine extension of the early-decision coverage beyond Alexander-only preprocessing.

7 Accelerating the Khovanov backend itself

7.1 The unchanged mathematical representation

The scanner attaches one crossing at a time. Objects are boundary matchings with a homological cube degree. A closed circle is delooped into two copies. For matchings a, b , glue a to the reflected b ; each resulting circle labels a dot variable. A morphism is a sum over $\mathbb{F}_2$ of square-free monomials in those variables, represented by a set of dot-subsets. This is the local dotted-cobordism method, with Gaussian cancellation after every attachment [3].

We retain this representation. In particular, isomorphic matching types are *not* merged into a single chain object: their multiplicities, cube degrees, and maps can carry essential homology. Quantum grading is forgotten because total rank suffices for recognition, but the ordinary chain-complex degree and differential are preserved.

7.2 Units are self-inverse over the square-zero algebra

For a matching with k arcs, its endomorphism algebra has the form

$$R_k = \mathbb{F}_2[x_1, \dots, x_k] / (x_1^2, \dots, x_k^2).$$

The unit test used by the scanner is the presence of the constant monomial.

Lemma 7.1. *Every element $u \in R_k$ with constant term 1 satisfies $u^2 = 1$.*

Proof. Write $u = 1 + v$, with v a sum of nonconstant square-free monomials. In characteristic two, all cross terms in v^2 vanish. Every remaining monomial square contains some x_i^2 and hence vanishes. Therefore $(1 + v)^2 = 1$. $\square$

The inverse routine consequently returns the same morphism instead of forming a nilpotent geometric series. The tests exhaust all 128 units of R_3 , using both the old and new multiplication. This shortcut depends on the coefficient field and relations; it must not be copied unchanged to integral or deformed Khovanov theories.

Multiplication in an endomorphism ring is also specialized: multiply two monomials only when their dot sets are disjoint, take their union, and XOR repeated outputs. Exact identity morphisms use an even shorter path. General source/intermediate/target matching triples still use the complete surface-composition routine.

7.3 Compile topology separately from coefficients

For $f : a \rightarrow b$ and $g : b \rightarrow c$, the connected surface components, their Euler characteristics, output boundary circles, and assignment of input dot-circles to components depend on (a, b, c) , not on the coefficients in f and g . The old implementation reconstructs this topology on every composition. The new `_composition_plan` caches it, after which each pair of monomials supplies only dot counts to the existing expansion rules. Repeated images of differential entries under the same crossing attachment are cached separately by `_crossing_entries`.

Four LRU caches have finite capacities of 16 384 entries each. Cached coefficient collections are immutable; the public crossing-entry wrapper returns mutable copies where mutation is needed. Caches are cleared at entry and in a `finally` block at exit from each nonempty rank calculation. This removes reliance on warm caches and prevents unbounded retention across unrelated jobs. Entry counts are not byte limits: an individual morphism or topological key can still be large. Internal cached geometry is treated as read-only by the engine.

7.4 Sparse pivot selection and Schur complements

Suppose a differential entry $\phi : b \rightarrow c$ is an isomorphism. For any other source $a \rightarrow c$ and target $b \rightarrow d$, cancellation replaces the corresponding map by

$$d'_{a,d} = d_{a,d} + d_{b,d} \phi^{-1} d_{a,c}. \quad (4)$$

The plus sign is subtraction over $\mathbb{F}_2$. In block form this is the Schur complement, obtained by invertible row and column operations followed by removing the contractible pair. Consequently pivot order affects work and fill-in but not homology.

The baseline takes available unit entries from a last-in-first-out list. The new engine uses the approximate Markowitz cost

$$(\# \text{ incoming}(c) - 1)(\# \text{ outgoing}(b) - 1),$$

namely the number of possible correction pairs in (4). A heap favors sparse pivots, especially zero-correction pivots. Entries are checked again when popped; stale or no-longer-invertible entries are discarded, and changed costs are reinserted. This is a *lazy heuristic*, not a claim that every chosen pivot globally minimizes the current fill cost. Stale keys of other heap entries may delay a better candidate.

The implementation factors $\phi^{-1}d_{a,c}$ once for each incoming source and reuses it for all outgoing targets. Lemma 7.1 removes inverse expansion. If either side is empty, no correction compositions are needed. The extra heap overhead can lose on easy cases; the benchmark measures rather than assumes a net gain.

The `compositions` statistic now counts actual composition calls. The baseline used a different counter convention, so cross-version ratios of that field would be misleading. Wall-clock time, computed ranks, maximum live objects, and scan boundaries have directly comparable meanings. The new `correction_pairs` and `cache-hit` fields are diagnostic only.

8 Incremental scan-order construction

Let s_i be the number of edges from a remaining crossing i into the processed region, and let ℓ_i be its number of self-loops. At a fixed step with boundary size B , the old greedy score is

$$(s_i, -(B + 4 - 2s_i - 2\ell_i), -i).$$

Lexicographically, this is equivalent to $(s_i, \ell_i, -i)$. The common B need not be recomputed for every candidate. A heap maintains the negated score; when a crossing is added, only its remaining neighbors need updates. Each nonloop edge raises a remaining neighbor's shared-edge count once. Old heap records are ignored when their shared count is stale.

Proposition 8.1. *For a fixed starting crossing, the new order is exactly the baseline's greedy order and takes $\mathcal{O}(n \log n)$ time and $\mathcal{O}(n)$ space instead of quadratic candidate rescanning.*

Proof. Score equivalence gives identical tie breaking. The crossing graph has bounded degree, so the total number of initial and updated heap records is $\mathcal{O}(n)$. Each extraction or insertion takes $\mathcal{O}(\log n)$ time. $\square$

The production calls preserve the baseline's approximate sampling policy with `tries=12`. This is a *target*, not a strict maximum number of starts: the inherited integer-step rule can sample as many as 23. It is nevertheless a fixed constant, so choosing the best sampled order still takes $\mathcal{O}(n \log n)$. Calling `best_scan_order(tries=None)` tries all n starts and instead takes $\mathcal{O}(n^2 \log n)$. The implementation and tests retain that public behavior.

Explicit orders now must be permutations of *all* crossing indices. The baseline did not enforce that condition; an empty supplied order on a nonempty input could bypass the entire computation. Duplicate, omitted, out-of-range, and boolean indices are rejected. This validation is important when experimenting with alternative order heuristics.

9 Complexity: the improvement and the remaining exponential

Stage	Supplied implementation	New implementation
R1/R2 cleanup	Repeated full searches and rebuilds; quadratic lower bound on curls	$\mathcal{O}(n \log n)$ word operations
Descending test	Quadratic worst-case traversal search	$\mathcal{O}(n)$ word operations
One greedy order	$\mathcal{O}(n^2)$ candidate work	$\mathcal{O}(n \log n)$ word operations
Determinant prefilter	Not a separate early stage	$\mathcal{O}(n^3)$ fixed-field operations
Jones prefilter	Absent	$n2^{\mathcal{O}(w \log w)}$ uncapped; bounded by work caps in production
Complete Kh fallback	Exponential	Exponential; less repeated algebra and fill

The $2^{\mathcal{O}(n)}$ envelope for the complete fallback is deliberately coarse. There are 2^n smoothing states, each with $\mathcal{O}(n)$ circles and hence at most $2^{\mathcal{O}(n)}$ enhanced states. Finite matrix operations on exponentially many generators remain within an exponential envelope over a fixed field. Polynomial preprocessing and bounded optional filters do not change that worst-case classification. No smaller exponential base is proved here.

Boundary width does not bound homology multiplicity. Counting boundary matchings alone is not a running-time proof for the Khovanov scanner. Consider a connected sum of m trefoils, arranged as a sequence of fixed-size tangle blocks. A left-to-right scan has constant frontier width. Over a field, the reduced connected-sum complex is the tensor product of the reduced complexes, so the reduced ranks multiply. The reduced trefoil rank is 3, and the sum has rank 3^m at $n = 3m$ crossings. An explicit minimal complex consequently has exponentially many generators despite constant width. The tensor-product assertion can also be seen directly by marking the connected-sum arc: after fixing its label in the reduced complex, smoothing choices and the differential split between the two factors; field-valued Künneth then multiplies ranks.

This example is about the explicit homology representation, not a lower bound for every unknot decision algorithm. Our modular determinant rejects that family cheaply. It explains why algebraic caching, width reduction, and geometric separator intuition cannot on their own establish a quasi-polynomial guarantee for the current fallback.

10 Validation and reproducible measurements

10.1 Correctness checks actually executed

The original suite passes all 19 tests against the unchanged baseline. The distributed suite has 37 passing tests. The legacy tests are retained; their pipeline-method assertions explicitly disable the two new filters, and the old resource-limit CLI check adds `-no-filters` so it still reaches the stage it is intended to test. This compatibility adjustment is documented rather than presented as an unmodified test suite.

The added checks include: exact scan-order comparisons on 100 generated knots and every possible starting crossing; 250 comparisons of complete simplification traces with independent replay; 300 comparisons of the linear descending test with the old brute-force search; and 400 morphism

composition comparisons on planar boundary matchings. All 128 units in the three-variable endomorphism algebra are tested against the old algebra.

The modular bracket is checked against an independent full state sum for 100 random knots at two evaluation points and arbitrary scan orders. The state-sum reference uses graph traversal rather than the frontier union-find routine. Other checks compare 120 determinant residues with the exact Alexander polynomial, test half-turns and mirrors on 80 inputs, check normalized bracket invariance under available R1/R2 moves on 80 inputs, and test 60 guaranteed unknots generated by braid conjugation and Markov stabilization. Mocked inconclusive residues verify that filter collisions cannot create a false unknot verdict.

For homology, every one-component three-strand braid word of lengths 2 and 4 over $\{\sigma_1^{\pm 1}, \sigma_2^{\pm 1}\}$ is checked: exactly 168 words. Their reduced ranks are compared with a separate dense reduced cube implementation. Another 100 randomly generated knots up to nine crossings receive the same comparison. The inherited tests include further small cube checks. Intermediate $d^2 = 0$ checks are enabled in the new exhaustive and random rank tests. Tests also cover malformed orders, invalid resource options, budget propagation, and cache cleanup.

Hard-case cross-check. The automatic greedy order and the natural input order both give reduced rank 2949 and identical unreduced ranks in every cube degree, with intermediate $d^2 = 0$ checks. The respective elapsed times in these debug runs are 2.89 and 19.36 seconds. Two exploratory alternative-order checks (reverse natural order and a greedy order starting at crossing 35) did not finish within the external run limits. No agreement is claimed for them. This illustrates important order sensitivity; these debug runs are not the three-repetition benchmark. The completed outputs are in `results/hard-case-verification.json`; the unsuccessful attempts are described in `results/additional-order-notes.txt`.

These are substantial regression checks, not a formal verification of the implementation or an exhaustive validation at 36 crossings. In particular, agreement between scan orders still shares the same cobordism algebra; the independent dense-cube checks cover small inputs, not this entire large case. Random tests use fixed seeds recorded in the source.

10.2 Methodology

The comparison uses the same interpreter and host for both packages: Python 3.13.5 on Linux-6.18.44-x86_64-with-glibc2.41. The reported processor is INTEL(R) XEON(R) PLATINUM 8573C. Each engine/input pair runs in a separate process, engines run sequentially, and their order alternates by case. Each successful measurement is the median of three calls with explicit algebra-cache clearing and garbage collection before the timed region. Imports and input construction are excluded. Raw samples and process peak RSS, when available, are retained in `results/benchmark.json`. RSS is a process high-water mark, not a per-call allocation measurement.

The corpus contains *all* 25 raw scan inputs and all 10 pipeline examples from the supplied benchmark, plus 10 larger preprocessing-only measurements. Every generated braid word is frozen in `tools/corpus.json`. Raw scan measurements bypass reductions, descending tests, and both old and new invariant filters. Pipeline measurements use the defaults of each version. Therefore a pipeline speedup and a raw homology speedup are not conflated.

Each call has a 30-second cooperative limit and a 500 000 Khovanov-object ceiling. A parent-process timeout backs up cooperative limits. A timed-out run is not assigned a fictitious completion time or a median of three successful runs: its first limit event is recorded and repetitions stop. Reported speedup lower bounds use the time limit divided by the optimized median. The original

archive’s Windows/Python timing of 600 seconds for the hard case is preserved as historical data, but is *not* used to compute a same-machine speedup.

10.3 Raw Khovanov results

Input family	n	Reduced rank	Old (s)	New (s)	Speedup
Stabilized unknot	8	1	0.0009	0.0009	1.03
Stabilized unknot	16	1	0.0029	0.0023	1.27
Stabilized unknot	32	1	0.0083	0.0036	2.32
Stabilized unknot	64	1	0.0223	0.0065	3.42
Stabilized unknot	128	1	0.0833	0.0156	5.34
Stabilized unknot	256	1	0.3008	0.0265	11.35
$T(2, k)$	5	5	0.0036	0.0016	2.27
$T(2, k)$	11	11	0.0142	0.0052	2.72
$T(2, k)$	21	21	0.0429	0.0151	2.85
$T(2, k)$	41	41	0.2163	0.0336	6.44
$T(3, k)$	8	5	0.0122	0.0082	1.48
$T(3, k)$	10	7	0.0133	0.0184	0.72
$T(3, k)$	14	9	0.0384	0.0261	1.47
$T(3, k)$	16	11	0.0374	0.0314	1.19
$T(3, k)$	20	13	0.0653	0.0617	1.06
$T(3, k)$	22	15	0.1708	0.0637	2.68
random 3-braid	20	75	0.2116	0.0719	2.94
random 3-braid	40	321	0.9623	0.2141	4.49
random 4-braid	15	5	0.0186	0.0089	2.08
random 4-braid	21	25	0.7309	0.1389	5.26
random 4-braid	31	45	0.3965	0.1255	3.16
random 4-braid	41	109	1.663	0.4447	3.74
random 5-braid	24	13	0.0670	0.0435	1.54
random 5-braid	36	2949	> 30	2.233	> 13.43
random 6-braid	31	85	1.008	0.1245	8.10

All successfully completed baseline/optimized comparisons agree on rank. The hard case has 2949 reduced generators and 5898 unreduced generators. Its optimized maximum object count before elimination is 17694, and its maximum frontier contains 8 edges. These counts emphasize that the improvement does not arise from secretly replacing the hard diagram by a lower-crossing representative in the raw benchmark. The scan order is the baseline’s own selected order.

10.4 Full pipeline results

Example	Old (ms)	New (ms)	Speedup	New decisive stage
Conway	47.693	1.622	29.41	Bracket
Figure eight	0.137	0.162	0.85	Determinant
Determinant-one grid	0.759	0.807	0.94	Bracket
Scrambled grid unknot	0.376	0.234	1.61	R1/R2
Hard unknot (8)	2.699	2.146	1.26	Khovanov
Kinoshita–Terasaka	48.491	1.004	48.28	Bracket
$T(3, 5)$	0.706	1.006	0.70	Bracket
Trefoil	0.117	0.153	0.76	Determinant
Unknot example	0.023	0.073	0.31	R1/R2
Unknot braid (40)	7.050	0.404	17.47	R1/R2

The measurements were made on a shared container host; scheduling noise can also produce non-monotone individual timings. No asymptotic exponent is fitted to these samples. The easiest examples may be slower because the new bookkeeping, input digest, and modular-stage dispatch have fixed overhead. Sub-millisecond differences should not be overinterpreted. The determinant-one examples are more informative: they demonstrate that the bracket stage can avoid the complete homology calculation on cases where the Alexander stage cannot decide. The supplied grid example is also retained rather than choosing only named low-crossing knots.

10.5 Preprocessing scaling

Subroutine	n	Old (s)	New (s)	Speedup
Scan ordering	128	0.0700	0.0026	26.6
R1/R2 cleanup	128	0.0733	0.0009	86.0
Scan ordering	256	0.2694	0.0052	52.1
R1/R2 cleanup	256	0.7270	0.0016	446.9
Scan ordering	512	1.088	0.0248	43.9
R1/R2 cleanup	512	1.107	0.0032	343.1
Scan ordering	1024	4.201	0.0211	198.8
R1/R2 cleanup	1024	4.883	0.0078	623.8
Scan ordering	2048	20.202	0.0531	380.3
R1/R2 cleanup	2048	25.057	0.0138	1818.4

These runs use the closure of $\sigma_1\sigma_2\cdots\sigma_n$ on $n + 1$ strands. For order construction, only `best_scan_order(tries=12)` is timed. For reduction, only the simplifier is timed, and both versions remove every crossing. The family has n crossings and linear PD size; braid construction is outside the timed region. The measurements illustrate the proved subroutine bounds, not a new asymptotic theorem about unknot recognition.

11 Using and extending the package

From the extracted archive root, Python 3.10 or later is sufficient. There are no runtime packages to install:

```
python -m fastunknot recognize examples/conway.json
python -m fastunknot khovanov examples/conway.json --check-d2
python -m unittest discover -s tests -v
python tools/benchmark.py --limit 30 --repeats 3
python tools/verify_hard_case.py
```

The installed `fastunknot` console command is also available after `python -m pip install ..` For a direct Python call:

```
from fastunknot import Diagram, recognize

diagram = Diagram.from_braid(2, [1, 1, 1])
result = recognize(diagram, seconds=30, max_objects=100_000)
print(result.status, result.method)
```

`-no-filters` disables both new modular stages while keeping the accelerated simplifier and Khovanov engine. `-no-alexander` disables only the full symbolic Alexander calculation; it does not disable the modular determinant. `-no-reduction` and `-no-descending` control the old positive-decision shortcuts. The `khovanov` subcommand bypasses every preprocessing stage. The `jones` subcommand reports a modular comparison, not a complete unknot verdict. Exit code 0 means a completed exact recognition result (either kind), 2 means invalid input or options, and 3 means a resource limit. Inspect the JSON status rather than treating every nonzero return as knottedness.

Time limits are cooperative. A large Python operation, input validation, order construction, or the inherited exact Alexander calculation can overshoot a requested deadline. The object ceiling counts chain objects, not bytes, dictionary entries, morphism terms, or cache payload. Untrusted large jobs should run in a separately supervised process with operating- system memory and wall-time limits. Cache clearing is safe for algebraic correctness but concurrent calls sharing a process can disrupt each other's cache-hit statistics and performance; the benchmark uses isolated sequential processes.

The highest-value next optimization experiment would be compact integer bitsets for dot monomials and interned local matching identifiers. It must preserve the coefficient/topology separation and the multiplicity of chain objects. Componentwise or connected-sum decomposition could avoid materializing some product homologies, but such a change needs a correct decomposition certificate and must not be inferred from repeated matching types alone. Neither direction is included in the claimed measurements or complexity improvements of this release.

12 Conclusion

The requested quasi-polynomial target is not established by this code. The fallback goal is achieved in two independent ways: several preprocessing subroutines have better proved asymptotic costs, and the complete supplied homology engine is substantially faster on the measured corpus. A previously unfinished 36-crossing homology computation now completes, while a bounded bracket stage provides fast certified rejection of determinant-one examples. The decisive distinction is between reducing repeated work and fill-in, which is implemented and measured here, and compressing the geometric

recognition problem enough to prove $n^{\mathcal{O}(\log n)}$, which requires a different set of invariants and algorithmic bounds.

References

- [1] M. Lackenby, *Unknot recognition in quasi-polynomial time*, talk slides, main theorem labeled 2021, 109 pages. Supplied as docs/quasipolynomial-talk.pdf. Online copy accessed 18 September 2026. <https://people.maths.ox.ac.uk/lackenby/quasipolynomial-talk.pdf>.
- [2] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026. Sections 9–10 and introduction. The supplied source is in docs/arXiv-2607.23350v1/. <https://arxiv.org/abs/2607.23350>.
- [3] D. Bar-Natan, *Fast Khovanov homology computations*, Journal of Knot Theory and Its Ramifications **16** (2007), 243–255. arXiv:math/0606318; delooping and Gaussian elimination are Lemmas 4.1 and 4.2 in the arXiv version. <https://arxiv.org/abs/math/0606318>.
- [4] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, Publications Mathématiques de l’IHÉS **113** (2011), 97–208. <https://arxiv.org/abs/1005.4346>.
- [5] A. N. Shumakovitch, *Torsion of the Khovanov homology*, arXiv:math/0405474v2, 2018. Corollary 3.2.C. <https://arxiv.org/abs/math/0405474>.
- [6] L. H. Kauffman, *State models and the Jones polynomial*, Topology **26** (1987), 395–407. [https://doi.org/10.1016/0040-9383\(87\)90009-7](https://doi.org/10.1016/0040-9383(87)90009-7).